JC3504 Robot Technology

Lecture 1: Introduction

Outline

- History of Robot
- Definition of Robot
- Course Arrangements

History of Robot

1495 UNIVERSITY OF ABERDEEN

Leonardo's Robot

Leonardo da Vinci's robot, conceptualised in the late 15th century, is considered one of the earliest attempts to create a mechanised human figure capable of movement. This visionary design, reflecting da Vinci's profound understanding of human anatomy and mechanics, showcases the remarkable foresight of automata's potential.

The Writer Automaton

Jaquet-Droz's "The Writer" is a remarkable example of 18th-century automaton craftsmanship, created by Pierre Jaquet-Droz, a Swiss watchmaker, and his son Henri-Louis. Completed around 1774, this automaton is one of the most famous and intricate mechanical devices of its time. "The Writer" is a mechanical boy seated at a desk, equipped with a quill pen that he dips into ink before writing any custom text up to 40 characters long on a piece of paper.

This feat is accomplished through a complex internal mechanism consisting of cams, levers, and gears, which can be programmed to follow specific patterns for each letter and number, allowing the automaton to write various phrases and messages. The device's ability to be programmed makes it a precursor to modern computers, showcasing the ingenuity and mechanical prowess of its creators.

The Writer Automaton

<video: 01 - The Writer.mp4>

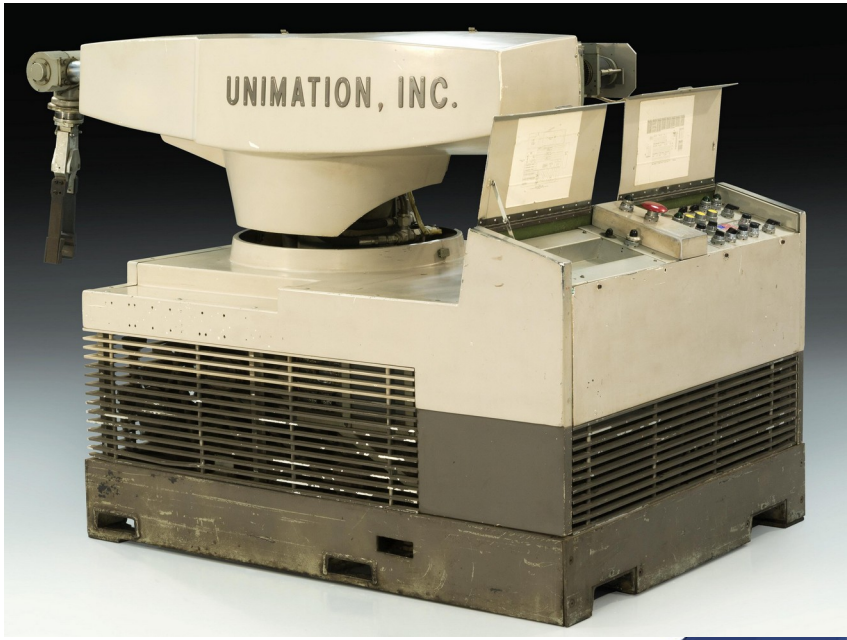
Mechanical Turk

The Mechanical Turk, also known as the Automaton Chess Player, or simply The Turk, was a fraudulent chess-playing machine constructed in 1770, which appeared to be able to play a strong game of chess against a human opponent. For 84 years, it was exhibited on tours by various owners as an automaton. The machine survived and continued giving occasional exhibitions until 1854, when a fire swept through the museum where it was kept, destroying the machine.

Afterwards, articles were published by a son of the machine's owner revealing its secrets to the public: that it was an elaborate hoax, suspected by some, but never proven in public while it still existed.

Unimate

George Devol's Unimate, introduced in 1961, is heralded as the world's first industrial robot, revolutionising manufacturing with its automated capabilities. This groundbreaking innovation laid the foundation for modern robotics, transforming production lines with unprecedented efficiency and precision.



Unimate

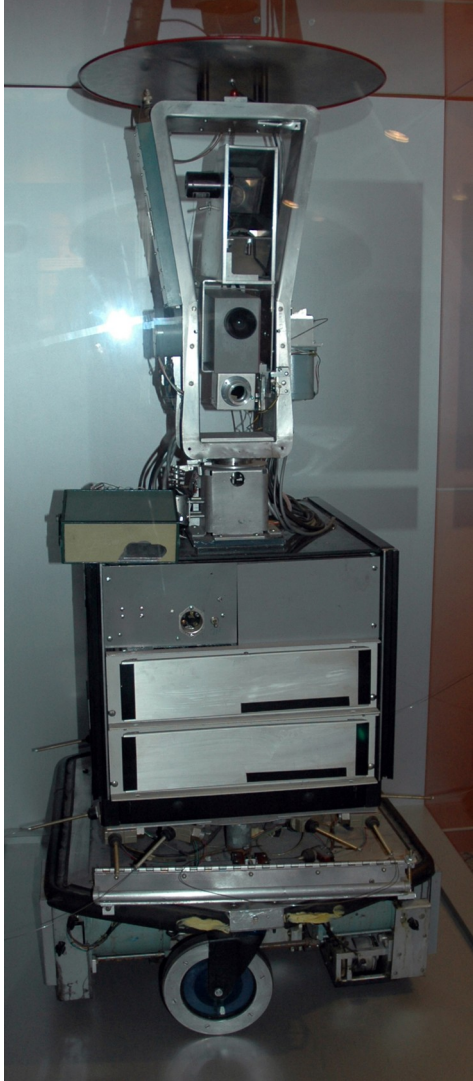
<Video: 01 - First Industrial Robot Ever.mp4>

Shakey

Shakey the Robot (1969) was the first general-purpose **mobile** robot able to reason about its own actions. While other robots would have to be instructed on each individual step of completing a larger task, Shakey could analyse commands and break them down into basic chunks by itself.

Due to its nature, the project combined research in robotics, computer vision, and natural language processing. Because of this, it was the first project that melded logical reasoning and physical action. Shakey was developed at the Artificial Intelligence Center of Stanford Research Institute (now called SRI International).

Some of the most notable results of the project include the A* search algorithm, the Hough transform, and the visibility graph method.



Shakey

<Video: 01 - Shakey the Robot.mp4>

Da Vinci Surgical System

The Da Vinci Surgical System represents a pioneering advancement in the field of robotic-assisted surgery, allowing surgeons to perform complex procedures with enhanced precision, flexibility, and control. Utilising a 3D vision system and tiny wristed instruments that bend and rotate far beyond the capabilities of the human hand, this innovative system has revolutionised minimally invasive surgery since its introduction.

Through its design, the Da Vinci Surgical System facilitates highly delicate operations, improving patient outcomes by reducing recovery times, minimising scarring, and lowering the risk of complications.

Amazon Warehouse Robots

The Amazon warehouse robots are an innovative automation system designed to enhance storage and sorting efficiency. These robots are engineered to navigate within the warehouse autonomously transporting goods based on order requirements. Equipped with advanced navigation technology and sensors, they can manoeuvre around obstacles and accurately locate the required items swiftly, Integrated with warehouse management systems, these robots provide real-time updates on the location of goods, making the entire storage process more efficient and intelligent.

Amazon Warehouse Robots

<Video: 01 - Inside the Amazon Robotic Sort Center in Tracy.mp4>

Boston Dynamics' Atlas and SpotMini

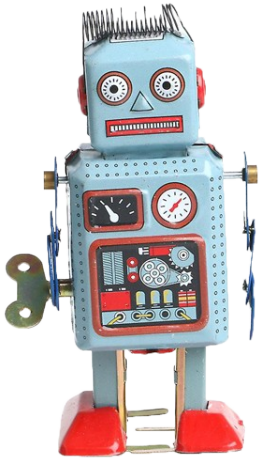
Boston Dynamics' Atlas and SpotMini are exemplary showcases of cutting-edge robotics technology, demonstrating great agility, mobility, and adaptability in robots. Atlas, often referred to as one of the world's most advanced humanoid robots, has been designed for a variety of search and rescue tasks. It exhibits remarkable human-like dexterity and mobility, capable of running, jumping over obstacles, and even performing backflips. On the other hand, SpotMini is a smaller, four-legged robot that resembles a dog. It's known for its ability to navigate complex environments, climb stairs, and handle objects with its mechanical arm. SpotMini's applications range from assisting in construction sites to performing tasks in offices and homes.

Boston Dynamics' SpotMini

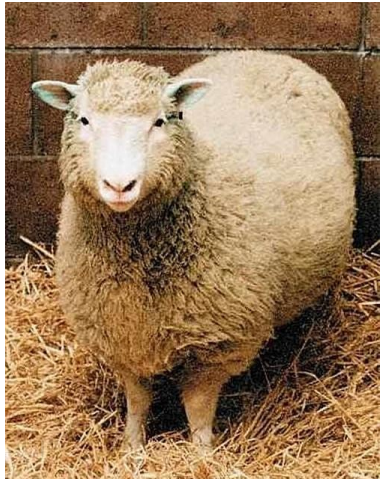
<Video: 01 - Boston Dynamics takes on its door-opening SpotMini.mp4>

What Is A Robot?

Are Them Robots?

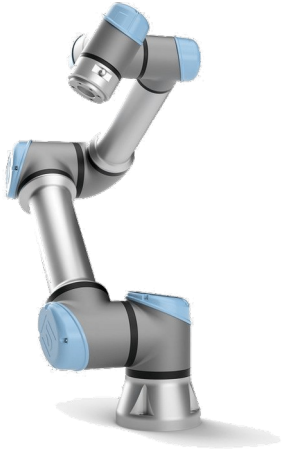


Are Them Robots?



ChatGPT

Are Them Robots?



Are Them Robots?

What Is A Robot?

A robot is a machine designed to execute one or more tasks automatically with speed and precision. There are several essential characteristics that define a robot:

- Autonomy -- Robots can operate on their own, making decisions based on pre-programmed instructions or through artificial intelligence (AI) without human intervention. The level of autonomy can vary, from fully autonomous robots that can navigate and make decisions independently, to semi-autonomous robots that require some level of human input.

What Is A Robot?

A robot is a machine designed to execute one or more tasks automatically with speed and precision. There are several essential characteristics that define a robot:

- Sensory Perception -- Robots often have sensors to perceive their surroundings. These can include cameras, microphones, temperature sensors, and tactile sensors, among others. Sensory input allows a robot to gather information about its environment, which it can then process to make decisions or take actions.

What Is A Robot?

A robot is a machine designed to execute one or more tasks automatically with speed and precision. There are several essential characteristics that define a robot:

- Actuation -- Robots react to their environment through actuators, which are the mechanisms that allow the robot to move or manipulate objects. This can include motors that drive wheels or legs for movement, arms for picking up or manipulating objects, or other tools specific to the robot's designed tasks.

What Is A Robot?

A robot is a machine designed to execute one or more tasks automatically with speed and precision. There are several essential characteristics that define a robot:

- Programmability -- Robots are programmable, meaning they can be coded to perform a wide variety of tasks. This programming can be fixed and unchangeable or adaptable, allowing the robot to learn from its environment and experiences to improve its performance over time.

What Is A Robot?

A robot is a machine designed to execute one or more tasks automatically with speed and precision. There are several essential characteristics that define a robot:

- Integration -- Robots can integrate the data received from their sensors to plan and execute tasks, navigate, or perform complex operations. This integration is often facilitated by sophisticated software that processes inputs and directs the robot's actions accordingly.

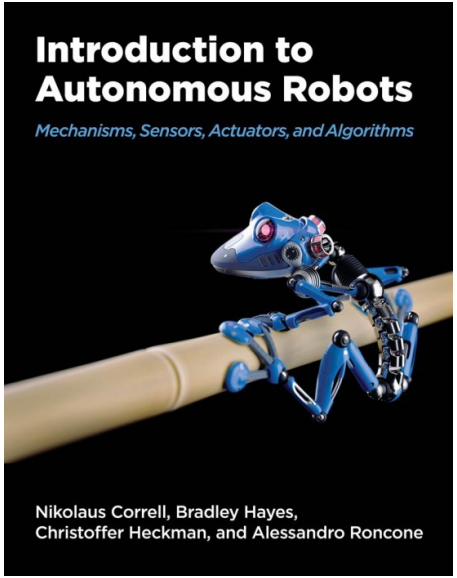
About Our Course

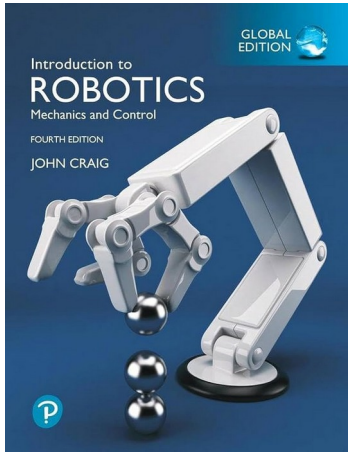
1495 UNIVERSITY OF ABERDEEN

Syllabus

- The Robot Operating System (ROS)
- Kinematic constraints on robot motion
- Overview of different sensing modalities
- Computer vision and image processing
- Localisation
- Planning and search
- Robot architectures

Textbook


Introduction to
Autonomous Robots
Mechanisms, Sensors, Actuators, and Algorithms
Nikolaus Correll, Bradley Hayes,
Christoffer Heckman, and Alessandro Roncone


GLOBAL EDITION
Introduction to
ROBOTICS
Mechanics and Control
FOURTH EDITION
JOHN CRAIG

HUMBLE
HAWKSBILL

ARTICLE
doi:10.1038/nature24270
Mastering the game of Go without human knowledge
David Silver*, Julian Schrittwieser*, Karen Simonyan*, Ioannis Antonoglou*, Aja Huang*, Arthur Guez*, Thomas Hubert*, Lucas Baker*, Matthew Lai*, Adrian Bolton*, Yutian Chen*, Timothy Lillicrap*, Fan Hui*, Laurent Sifre*, George van den Driessche*, Thore Graepel* & Demis Hassabis*
A long-standing goal of artificial intelligence is an algorithm that learns, tabula rasa, superhuman proficiency in challenging domains. Recently, AlphaGo became the first program to defeat a world champion in the game of Go. The tree search in AlphaGo evaluated positions and selected moves using deep neural networks. These neural networks were trained by supervised learning from human expert moves, and by reinforcement learning from self-play. Here we introduce an algorithm based solely on reinforcement learning, without human data, guidance or domain knowledge beyond game rules. AlphaGo becomes its own teacher: a neural network is trained to predict AlphaGo's own move selections and also the winner of AlphaGo's games. This neural network improves the strength of the tree search, resulting in higher quality move selection and stronger self-play in the next iteration. Starting tabula rasa, our new program AlphaGo Zero achieved superhuman performance, winning 100–0 against the previously published, champion-defeating AlphaGo.
Much progress towards artificial intelligence has been made using supervised learning systems that are trained to replicate the decisions of human experts[1–5]. However, expert data sets are often expensive, unreliable or simply unavailable. Even when reliable data sets are available, they may impose a ceiling on the performance of systems trained in this manner[6]. By contrast, reinforcement learning systems are trained from their own experience, in principle allowing them to learn beyond human knowledge and to discover new forms of strategic expertise is lacking. Recently, there has been rapid progress towards this goal, using deep neural networks trained by reinforcement learning. These systems have outperformed humans in computer games, such as Atari[7–9] and 3D-virtual environments[10–12]. However, the most challenging domains in terms of human intellect—such as the game of Go, widely viewed as a grand challenge for artificial intelligence[13]—require a precise and sophisticated lookahead in vast search spaces. Fully general methods have not previously achieved human-level performance in these domains.
AlphaGo was the first program to achieve superhuman performance in Go. The published version[14], which we refer to as AlphaGo Fan, defeated the European champion Fan Hui in October 2015. AlphaGo Fan used two deep neural networks: a policy network that outputs move probabilities and a value network that outputs a position evaluation. The policy network was trained initially by supervised learning to accurately predict human expert moves, and was refined by policy-gradient reinforcement learning. The value network was trained to predict the winner of games played by the policy network against itself. Once trained, these networks were combined with a Monte Carlo tree search (MCTS)[15–17] to provide a lookahead search, using the policy network to narrow down the search to high-probability moves, and using the value network (in conjunction with Monte Carlo rollouts using a fast rollout policy) to evaluate positions in the tree. A subsequent version, which we refer to as AlphaGo Lee, used a similar approach (see Methods), and defeated Lee Sedol, the winner of 18 international titles, in March 2016.
Our program, AlphaGo Zero, differs from AlphaGo Fan and AlphaGo Lee[14] in several important aspects. First and foremost, it is trained solely by self-play reinforcement learning, starting from random play, without any supervision or use of human data. Second, it uses only the black and white stones from the board as input features. Third, it uses a single neural network, rather than separate policy and value networks. Finally, it uses a simpler tree search that relies upon this single neural network to evaluate positions and sample moves, without performing any Monte Carlo rollouts. To achieve these results, we introduce a new reinforcement learning algorithm that incorporates lookahead search inside the training loop, resulting in rapid improvement and precise and stable learning. Further technical differences in the search algorithm, training procedure and network architecture are described in Methods.
Reinforcement learning in AlphaGo Zero
Our new method uses a deep neural network f_θ with parameters θ. This neural network takes as an input the raw board representation s (see Methods), and outputs both a vector $\mathbf{p}$, representing a probability for each move a ($\mathbf{p}_a = \Pr(a|s)$), and a scalar value v, estimating the probability of the current player winning from position s. This neural network combines the roles of both policy network and value network[14] into a single architecture. The neural network consists of many residual blocks[4] of convolutional layers[18,19] with batch normalization[20] and rectifier nonlinearities[21] (see Methods).
The neural network in AlphaGo Zero is trained from games of self-play by a novel reinforcement learning algorithm. In each position s, an MCTS search is executed, guided by the neural network f_θ. The MCTS search outputs probabilities $\boldsymbol{\pi}$ of playing each move. These search probabilities usually select much stronger moves than the raw moves of the neural network $\mathbf{p}$ (see MCTS in Methods); MCTS may therefore be viewed as a powerful policy improvement operator[22,23]. Self-play with search—using the improved MCTS-based policy to select each move, then using the game winner z as a sample of the value—may be viewed as a powerful policy evaluation operator. The main idea of our reinforcement learning algorithm is to use these search operators
DeepMind, 5 New Street Square, London EC4A 3TW, UK.
*These authors contributed equally to this work.
334 | NATURE | VOL 550 | 19 OCTOBER 2017
© 2017 Macmillan Publishers Limited, part of Springer Nature. All rights reserved.

Teaching Arrangements

Teaching Arrangements:

- 32 theory sessions (across 16 weeks)
- 32 practical sessions (across 16 weeks)

Assessment:

- Group programming assignment (50%) x 2

Theory Session Arrangements

1. Introduction
2. The Robot Operating System (ROS)
3. Mechanical Manipulator Introduction
4. Kinematics (1)
5. Kinematics (2)
6. Sensing and Actuation
7. Computer Vision (1)
8. Computer Vision (2)
9. Computer Vision (3)
10. [Extension] Biology and Robotics
11. Plan and Search
12. Localisation
13. Simultaneous Localisation and Mapping
14. Reinforcement Learning (1)
15. Reinforcement Learning (2)
16. Robot System Integration & Architectures

Practical Session Arrangements

For our practical sessions, we will dedicate our efforts entirely to learning ROS. Tomorrow's lecture will provide a basic introduction to ROS, then, subsequent practical sessions will be geared towards self-guided learning of ROS. We will provide a series of practical guides to step-by-step familiarise you with ROS.

The content of the lectures may not align with ROS, as some topics covered in the lectures can be challenging to implement directly with ROS. Thus, you are encouraged to tailor your practical sessions according to your personal preferences and pace.

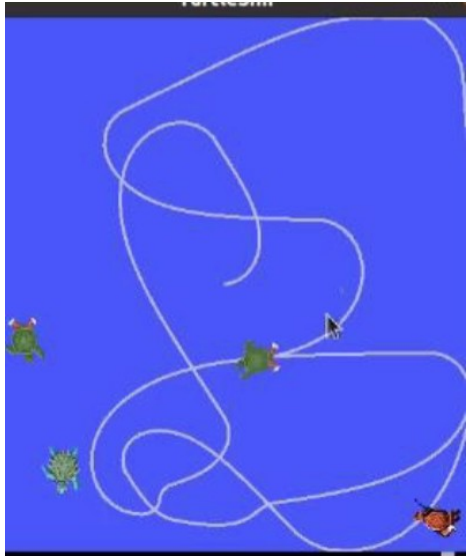
By the end of this four-week course, we hope you will have acquired a fundamental proficiency in using ROS.

You are encouraged to utilise the practical sessions to complete your project assignments.

Assessment 1 -- Catch Turtle All

- In this project, you are required to develop and implement an interactive simulation in the Turtlesim environment using ROS2.
- Your main objective is to program a master turtle to "catch" other turtles that appear randomly within the environment.

See: Catch Turtle All.pdf



Assessment 1 -- Catch Turtle All

Group Project:

- 3-5 people/group

Deadline:

- 11 May 2025

Submission:

- Code
- Video Evidence !
- Project Report
- Group workload profile

See: Catch Turtle All.pdf

Assessment 2 – Robert Development Proposal

- Complete a proposal for a robot project design. The proposal requirements are given in **Proposal Assessment.pdf**.
- You need to fill in the questions in **Proposal Assessment.pdf** according to the requirements to complete the proposal.

Group Project:

- 3-5 people/group

Deadline:

- 1 June 2025

Submission:

- Completed Proposal (pdf / docx)

Policy & Procedures on Extensions and Penalties

Appendix A - Policy and Procedures on Extensions and Penalties for Unauthorised Late Submissions of Coursework.docx

Students requesting **extensions** – email SCNU Enquiries: uoa-ji-enquiries@abdn.ac.uk

Penalties for Unauthorised Late Submissions:

1. **Up to 24 hours late:** the grade will be deducted by 2 CGS points (1 CGS = 4%);
2. **For each subsequent day:** up to a maximum of seven days, the grade will be deducted by a further CGS point for each day, or part of a day, up to a maximum of seven days late;
3. If you only save your assignment as a draft, it will be considered a seven-day delay.
4. **Over seven days late:** a grade of **G3** will be awarded.
5. If your assignment cannot be read, it will be graded as G3.

UNIVERSITY OF ABERDEEN

Ask Questions

We encourage students to ask questions at any time during working hours.

- Just Ask: feel free to interrupt the me with your questions without the need to raise your hand, as we promote discussion in the classroom.
- You can also make use of the time during lab sessions or breaks to ask questions.
- Additionally, you are welcome to send emails with your queries, or pose them during the Office Hour on Thursday afternoons (it would be prudent to check the lecturer availability in advance due to the limited office space).

No Code Show Policy: We will NOT look at students' code during as students are expected to know how to debug issues with their own code.